

=====

SmartSense - Report

=====



G705 – IES 2025/26

Project Team:

Margarida Cardoso - 125799 - Team Manager

Tiago Costa - 125943 - Product Owner

Pedro Rocha - 125296 - Architect

Daniel Martins - 115868 - DevOps Master

Index:

Index:	1
1 Executive summary.....	2
2 Introduction	3
3 Requirements analysis.....	4
3.1 Personas	4
3.2 System Requirements	4
3.2.1 Functional Requirements (FR)	4
3.2.2 Non-Functional Requirements (NFR)	5
3.3 User Stories	5
4 Architecture and Design	6
4.1 System Layers.....	6
4.1.1 Data Acquisition Layer (Sensor data).....	6
4.1.2 Business Logic Layer (Backend).....	6
4.1.3 User Interface Layer (UI).....	7
4.2 Data Design and UML Diagram	9
5 API Documentation.....	9
6 Implementation details	9
6.1 Backend Implementation (Spring Boot)	10
6.1.1 Core Technologies	10
6.1.2 Key Service Modules.....	10
6.2 Frontend Implementation (React)	10
6.2.1 UI Stack.....	10
6.2.2 Dashboard Architecture.....	11
6.3 Sensor Simulation (Python)	11
6.4 Infrastructure and Deployment.....	11
7 Inside SmartSense.....	11
7.1 Common Elements	12
7.2 Technician Elements	12
7.3 Director Elements	12
7.4 Administrator Elements	13

8	Conclusion	13
8.1	Objectives Achieved	13
8.2	Pros (Advantages)	14
8.3	Cons (Limitations)	14

1 Executive summary

This project, developed for the IES 2025/2026 course, presents an integrated Industrial Maintenance and Monitoring System designed to enable predictive maintenance of machinery, optimizing factory operations and minimize equipment downtime. Built using a Java Spring Boot backend and a React-based frontend, the platform provides a centralized hub for real-time machine tracking and maintenance management.

A core feature of the system is the support of a dedicated sensor publisher that simulates real-time data streams. This enables the platform to offer proactive breakdown detection and automated assistance requests. The application serves multiple organizational roles, providing a personal dashboard for every user. By connecting machine sensor data to management processes, the system makes communication faster, increases work efficiency, and provides clear, useful data through maintenance statistics and live monitoring.

2 Introduction

In today's industrial landscape, transitioning from a purely reactive maintenance approach to predictive strategies is essential for maximizing manufacturing efficiency. SmartSense was developed with this purpose, consisting of an industrial monitoring system designed to analyse data from vibration, pressure, and temperature sensors, enabling the predictive maintenance of machinery.

The main objective of the project is to mitigate equipment downtime through proactive anomaly detection. To achieve this, the platform integrates a telemetry data pipeline, supported by a real-time sensor simulator, which allows the identification of deviations from normal operating thresholds. By combining this data acquisition with management tools, the system ensures:

- **Real-Time Monitoring:** Constant tracking of machine health and metrics (vibration, pressure, temperature).
- **Predictive Action and Fault Detection:** Early identification of critical conditions, allowing for a quicker detection and maintenance.
- **Targeted Operational Optimization:** Specific interfaces and logics for each role. This allows Technicians to manage repairs faster, Directors to coordinate the team, and Administrators to manage accounts and assets.

In this way, the project establishes a direct bridge between the physical reality of the factory floor and administrative maintenance management, promoting agile communication, work efficiency, and decisions based on statistical data and real-time status.

3 Requirements analysis

The requirements analysis defines the functional and technical specifications for the SmartSense platform. To align the system with operational needs, this section details the creation of user personas, outlines the core system requirements, and defines the user stories that guide the development of the monitoring and predictive maintenance features.

3.1 Personas

Role	Technician	Director	Administrator
Name	João Neves	Manuel Gomes	Sara Lopes
Needs	Quick identification of faulty machines, access to historical data, and real-time telemetry for accurate diagnosis.	A holistic view of failure history, pattern identification, and coordination of assistance requests among technicians.	Monitoring team productivity, managing machine inventory (registration/removal), and access to historical archives.
Motivation	Professional recognition and efficiency in problem-solving.	Operational efficiency, team safety.	Continuous improvement of operations.
Photo			

3.2 System Requirements

3.2.1 Functional Requirements (FR)

- **FR1:** Support for different types of users (Technician, Director, Admin).
- **FR2:** Secure user authentication.

- **FR3:** Role-based access control.
- **FR4:** Storage of machine data (ID, status, location).
- **FR5:** Updating machine information.
- **FR6:** Sensor data collection (Vibration, Pressure, Temperature).
- **FR7:** Storage of telemetry history.
- **FR8:** Automatic generation of issue logs when a failure is detected.
- **FR9:** Registration of maintenance activities (pending and completed).
- **FR10:** Assistance request mechanism for technicians.

3.2.2 Non-Functional Requirements (NFR)

- **NFR1:** Near real-time processing of sensor data.
- **NFR2:** Simultaneous monitoring of multiple machines.
- **NFR3:** Integrity of collected data.
- **NFR4:** Secure authentication and credential protection.
- **NFR5:** Intuitive and responsive interface.
- **NFR6:** Scalability for new assets without loss of performance.
- **NFR7:** Extensibility for future monitoring technologies.

3.3 User Stories

	User Story
US1	As a Maintenance Technician, I want to know if there is a machine with a breakdown, So that I can fix it.
US2	As a Maintenance Technician, I want to monitor early warning signs from the machines, So that I can perform preventive repairs before a total breakdown occurs.
US3	As a Maintenance Technician, I want to view each machine's priority level based on its importance, downtime, and fault severity, So that I can optimize my workflow and fix the most critical issues first.
US4	As a Maintenance Technician, I want to request assistance for a machine in the app when a repair requires additional help, So that my colleagues are notified and can assist me.
US5	As a Maintenance Director, I want to know if there is a machine that has been having several breakdowns, So that I can investigate a more serious fault.
US6	As a Maintenance Director, I want to choose the technicians that can help when there is a request

	sent by another technician, So that the machine can be fixed quicker.
US7	As an Administrator, I want to monitor the technicians' performance, So that I can ensure the maintenance team is meeting productivity expectations.
US8	As an Administrator, I want to register new equipment/machines in the app, So that the maintenance team can start tracking its performance and history.
US9	As an Administrator, I want to remove equipment/machines from the app, So that the views of every user remain updated.
US10	As an Administrator, I want to remove equipment/machines from the app, So that the views of every user remain updated.

4 Architecture and Design

SmartSense adopts a distributed layered architecture designed to support high-frequency data ingestion and provide a reactive management interface. The solution is composed of three main layers: Data Acquisition (Sensor data supported by a Message Queue), Business Logic (Backend Spring Boot), and User Interface (Frontend).

4.1 System Layers

4.1.1 Data Acquisition Layer (Sensor data)

This layer is responsible for bridging the physical world (simulated) and the digital system.

- **Components:** Python scripts (sensor_publisher.py) act as local agents.
- **Operation:** Sensors generate continuous readings for Vibration (Hz), Pressure (bar), and Temperature (°C). Data is aggregated and forwarded to the message queue, which publishes it and injects it onto the Messaging sub-layer of the Backend main layer.
- **Anomaly Detection:** The gateway simulates safety thresholds, sending data that can trigger automatic alerts within the system when values exceed normal operating limits.

4.1.2 Business Logic Layer (Backend)

The engine of SmartSense, built with **Java Spring Boot**, processes and stores all system intelligence.

- **Internal Structure:**
 - **Controllers:** Expose the REST API for both the frontend and the sensors.
 - **Services:** Implement business rules, such as machine priority calculation and maintenance state management. Services act as the business logic layer, orchestrating operations between controllers and repositories to ensure the integrity of domain rules.
 - **Repositories:** Handles reading and writing data between the application and the database or storage layer.
 - **Domain:** Defines the data model (Machines, Problems, Readings, Users, etc).
 - **Mapper:** Converts data between different object types, such as entities and DTOs.
 - **Messaging:** Used as a two-ended gateway, linking the RabbitMQ component to the backend, allowing data to flow back and forth between the message queue and the backend.
 - **DTO:** A data container used to move information between the backend and frontend layers without exposing the internal domain entities directly.
- **Security:** JWT (JSON Web Token) implementation for authentication and Role-Based Access Control (RBAC).

4.1.3 User Interface Layer (UI)

Developed in **React** focuses on data visualization and user experience.

- **Reactive Dashboards:** Utilization of Recharts for visualizing telemetry trends.
- **Componentization:** Interface based on reusable components (Shadcn/ui), ensuring a consistent view across different profiles (Technician vs. Director).
- **Global State:** Management of authentication and machine data via the Context API.

It's split up into 3 sub-layers:

- **Technician Interface:** Contains the logic related to technicians using the application, including their profiles, assigned machines, requests made by directors, and current maintenance tasks.

- **Director Interface:** Contains the logic related to directors using the application, including their profiles, main dashboard, machines, machine assignment forms, requests, and team activity.
- **Admin Interface:** Contains the logic related to admins using the application, including their profiles, main dashboard, machines and team of active technicians.

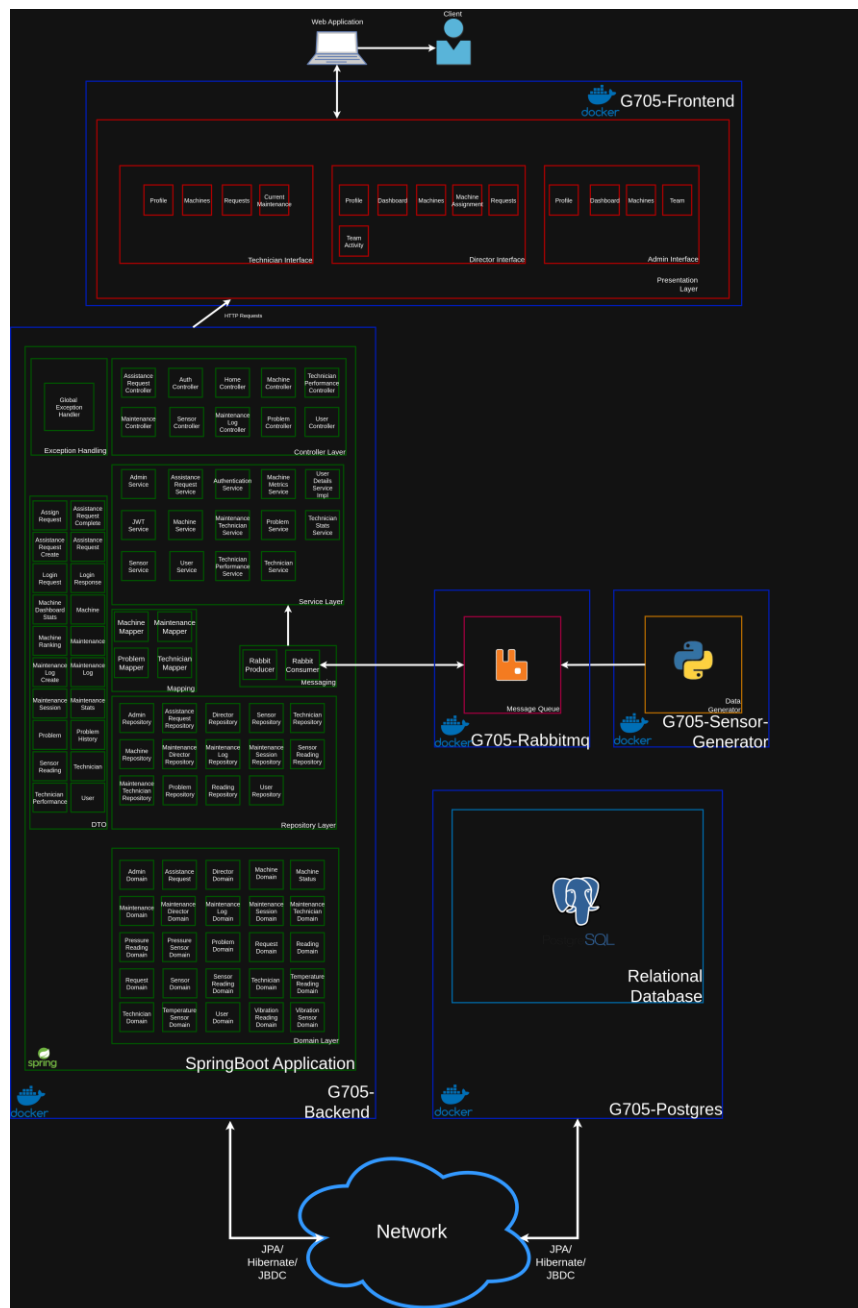


Image path: ./projX/docs/images/architecture/architectureNew.png

4.2 Data Design and UML Diagram

The data design was structured to support user profile inheritance and dynamic association of sensors to machines.

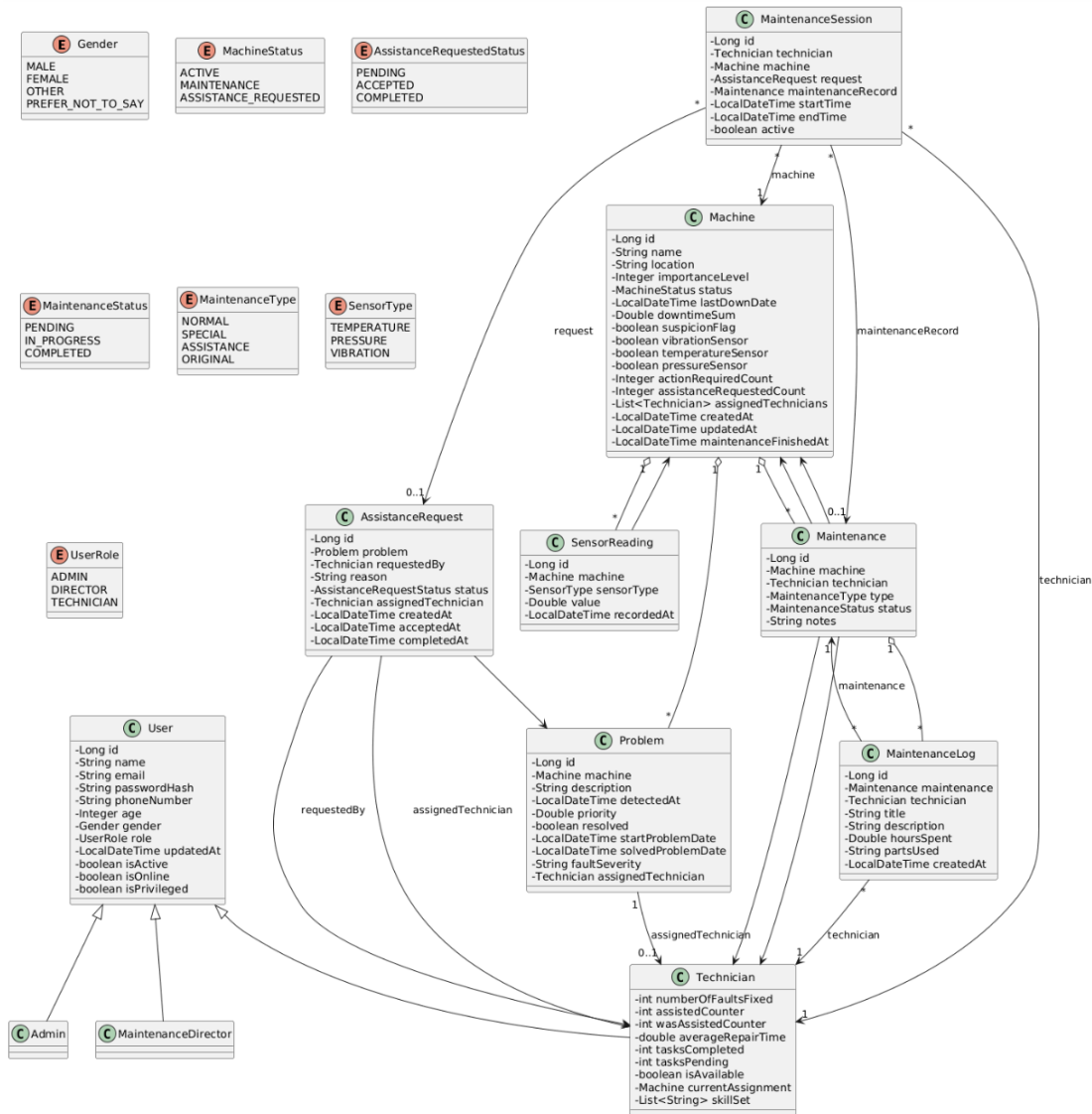


Image path: ./projX/docs/images/uml/UML.png

5 API Documentation

Full API documentation is available in a separate document inside the repository, with every endpoint explained. It is also documented in Swagger.

Path for the document: ./projX/docs/api-documentation/API_documentation.pdf

6 Implementation details

6.1 Backend Implementation (Spring Boot)

The core of the system is a Java 17 Spring Boot application, following a hexagonal-inspired layered architecture.

6.1.1 Core Technologies

- **Framework:** Spring Boot 3.2.2.
- **Data Persistence:** Spring Data JPA with a PostgreSQL production database.
- **Security:** Spring Security with **JWT (JSON Web Tokens)** for stateless authentication.
- **Messaging:** Spring AMQP (RabbitMQ) for asynchronous telemetry ingestion.
- **API Documentation:** SpringDoc OpenAPI (Swagger UI).

6.1.2 Key Service Modules

- **Machine Management:** Handles the lifecycle of industrial assets. It includes logic for status transitions (e.g., from ACTIVE to MAINTENANCE) and importance-based prioritization.
- **Telemetry Processing:** Implemented via `SensorService` and `RabbitConsumer`. The system listens to a dedicated RabbitMQ queue (`my.queue`) to consume sensor data asynchronously, ensuring the API remains responsive even under high load.
- **Predictive Logic:** The system monitors incoming data against safety thresholds. When a reading exceeds normal limits, a `Problem` entity is automatically instantiated, and the machine's state is updated.
- **Role-Based Access Control:** Custom filters (`JwtFilter`) intercept requests to validate tokens and enforce permissions for Technician, Director, and Admin roles.

6.2 Frontend Implementation (React)

The frontend is a reactive Single Page Application (SPA) built with **React 19** and **Vite**, prioritizing a clean and efficient industrial user experience.

6.2.1 UI Stack

- **Styling:** Tailwind CSS for responsive and utility-first design.
- **Components:** Built on top of Radix UI primitives (via `Shadcn/ui`), providing high accessibility and consistency.

- **Data Visualization: Recharts** is used to render live telemetry data (vibration, pressure, temperature), allowing technicians to spot trends visually.
- **Icons & Motion:** Lucide React for icons.

6.2.2 Dashboard Architecture

- **Role-Specific Views:** The UI dynamically renders sidebars and components based on the user's role stored in the AuthContext.
- **Real-time Updates:** The frontend periodically polls or receives updates to reflect the current status of the factory floor, ensuring technicians see, for example, "Suspicious" flags immediately.

6.3 Sensor Simulation (Python)

To demonstrate the system's capabilities without requiring physical hardware, a **Sensor Publisher** was implemented in Python.

- **Functionality:** It simulates a "Site Gateway" that aggregates data from multiple sensors attached to the machines.
- **Data Generation:** Uses statistical models to generate realistic fluctuations in vibration, pressure, and temperature.
- **Integration:** Communicates with the backend via REST/RabbitMQ, mimicking a real-world IoT deployment where bandwidth-savvy protocols are preferred.

6.4 Infrastructure and Deployment

The entire ecosystem is containerized using **Docker**, facilitating consistent environments across development and production.

- **Docker Compose:** Orchestrates multiple containers including the Spring Boot App, PostgreSQL database, RabbitMQ broker, the React frontend and the Sensor Generator Infrastructure.
- **Configuration:** Managed through Spring Profiles (application-prod.properties), allowing seamless transitions between local development and production-ready deployments.
- **Data Initialization:** A DataInitializer component populates the system with baseline machines, sensors, and users upon the first startup to ensure the system is ready for immediate demonstration.

7 Inside SmartSense

The SmartSense interface transforms sensor data into clear, actionable information. By using a role-based access control model, the application adapts the environment to the specific needs of each user profile.

While common navigation elements ensure visual consistency, specialized modules provide technicians, directors, and administrators with the exact tools required for their daily tasks. This section details the internal structure of the application, covering everything from shared monitoring dashboards to high-level management views.

7.1 Common Elements

- **Login** – Web interface where users enter their login credentials.
- **Individual Machine Dashboard** – Web interface displaying comprehensive machine data, including real-time sensor values, issues, and maintenance logs.

7.2 Technician Elements

- **Machines Tab** – Web interface displaying all machines assigned to the technician. Here, technicians can analyse which machines are operating normally, under maintenance, or flagged as suspicious.
- **Current Maintenance Tab** – Web interface showing the technician's active maintenance tasks. Technicians can complete maintenance by submitting a form.
- **Assistance Requests Tab** – Web interface displaying assistance requests assigned by directors. Technicians can complete requests and directly access the corresponding machine dashboard.
- **Profile Tab** – Web interface where technicians can update their personal information and view work-related statistics.

7.3 Director Elements

- **Profile Tab** – Web interface where directors can update their personal information.
- **Dashboard Tab** – Web interface providing real-time system overview with machine status, recent breakdowns, and pending assistance requests.
- **Machines Tab** – Web interface displaying all machines under the director's supervision. Directors can view machine statuses (Normal, Under Maintenance, or Suspicious).

- **Machine Assignment Tab** – Web interface where directors can assign technicians to machines, enabling them to view those machines in their Machines tab.
- **Assistance Requests Tab** – Web interface where directors can view pending, in-progress, and completed assistance requests, and can assign or remove technicians from requests.
- **Team Activity Tab** – Web interface where directors can monitor their team's activities in real time and view overall performance statistics.

7.4 Administrator Elements

- **Profile Tab** – Web interface where administrators can update their personal information.
- **Dashboard Tab** – Web interface providing real-time system overview with machine status, recent breakdowns, and pending assistance requests.
- **Machines Tab** – Web interface where administrators can add, remove, edit, and view machine details.
- **Team Tab** – Web interface where administrators can add, remove, and edit user accounts for directors and technicians.

8 Conclusion

The development of SmartSense successfully demonstrated the viability and value of an Integrated Industrial Monitoring and Maintenance System focused on transitioning from a purely reactive maintenance paradigm to a predictive strategy. Through a robust distributed architecture—combining a Java Spring Boot backend, a reactive React frontend, and a Python-based sensor simulation infrastructure—the project's core objectives were fully achieved. The built platform acts as an effective link between the shop floor and administrative management, centralizing information and streamlining decision-making.

8.1 Objectives Achieved

- **Real-Time Monitoring:** Implementation of a continuous data stream of sensor data (vibration, pressure, and temperature) efficiently processed through RabbitMQ.
- **Predictive Action and Anomaly Detection:** The system can identify deviations from normal operating parameters and automatically generating failure records before a critical equipment failure occurs.
- **Targeted Operational Optimization:** The application of a Role-Based Access Control model allowed the development of dashboards perfectly

tailored to the needs of Technicians, Directors, and Administrators, improving workflow and service request coordination.

8.2 Pros (Advantages)

- **Downtime Reduction:** The ability to predict failures and alert the team in advance allows for timely interventions, increasing the profitability of industrial operations.
- **Scalable and Modular Architecture:** The use of Docker containers and the clear separation into layers (Data Acquisition, Business Logic, and Interface) facilitate code maintenance and the future addition of new assets or monitoring technologies.
- **Intuitive and Focused Interface:** The use of modern UI libraries (Tailwind CSS, Shadcn) and interactive charts (Recharts) provides an excellent user experience, making complex data reading more accessible.

8.3 Cons (Limitations)

- **Connectivity Dependence:** Since anomaly detection relies on continuous telemetry analysis, a network failure or communication interruption with the Message Queue can delay alerts for critical issues.
- **Simulated Sensor Environment:** The project currently uses Python scripts to generate sensor data. Transitioning to a real-world scenario will require integration with physical industrial hardware and protocols, which may introduce challenges such as data noise or calibration issues.

In summary, the SmartSense project strictly fulfills the established functional and non-functional requirements, constituting a solid technological solution for reactive maintenance systems.